MODULE 09 · DOMAIN 2 - CONFIGURATION MANAGEMENT AND IAC

CloudFormation for Operators

Change sets and drift, protecting data from replacement and deletion, bootstrapping instances with signals, custom resources, and reshaping stacks.

DOMAIN 2 of 6 EXAM WEIGHT 17% MODULE 9 of 34

admin@sthithapragna.com

A stack is a **promise** that the template and reality match. Operating CloudFormation means managing the moments that promise is at risk.

Moments of risk, and the tool for each

THE MOMENT	THE TOOL
About to change something	Change sets
Someone changed it by hand	Drift detection; drift-aware change sets
An update would replace or delete data	Update replace and deletion policies; stack policy
Instance up, software not ready	Creation policy with cfn-signal
An update fails part-way	Rollback, rollback triggers, continue rollback
Existing resources, or the wrong structure	Import, IaC generator, stack refactoring

ON THE EXAM

Match the moment in the scenario to its row, and most options belong to a different row.

Change sets: look before you leap

- A change set previews what an update will add, modify or remove, **before** anything happens.
- Each modification shows whether it needs **replacement**: True, False, or Conditional.
- In a pipeline: create change set, manual approval, execute change set, as three actions.
- A change set shows what CloudFormation will do, not whether the application will work.

> ON THE EXAM
>
> Review infrastructure changes before production, least overhead: change set, approval, execute, in the pipeline.

Three kinds of update

UPDATE BEHAVIOUR	WHAT HAPPENS	WATCH OUT FOR
No interruption	Changed in place, keeps serving	Nothing
Some interruption	Changed in place, briefly unavailable	An instance reboot
Replacement	New resource created, old one deleted	New ID, new endpoint, lost data

ON THE EXAM

Replacement of a stateful resource is how data is lost. A change set showing Replacement: True on a database is a stop sign.

Deletion policy, or update replace policy?

DELETIONPOLICY

When the resource leaves the stack

- Removed from the template, or stack deleted
- Delete, Retain, Snapshot
- RetainExceptOnCreate: clean up failed creates
- Protects against deletion

UPDATEREPLACEPOLICY

When an update replaces it

- The old copy, after a replacement
- Delete, Retain, Snapshot
- RetainExceptOnCreate is not allowed
- Protects against replacement

THE TIEBREAKER

Many stateful resources need both. A deletion policy alone does nothing when an update replaces the resource.

Stopping a dangerous update before it happens

- A **stack policy** denies update actions on named resources, such as Update:Replace or Update:Delete.
- It applies to **everyone** who updates the stack, including pipelines, and can be overridden deliberately.
- **Termination protection** blocks deleting the whole stack. It does nothing for updates.
- Once set, a stack policy cannot be removed, only replaced. The default after setting one is deny.

> ON THE EXAM
>
> Prevent any update from replacing the production database, for every user: a stack policy on that resource.

Drift: when reality and template disagree

- **Drift detection** compares live resources with the template: IN_SYNC, MODIFIED, DELETED, NOT_CHECKED.
- The Config managed rule **cloudformation-stack-drift-detection-check** runs it continuously.
- Classic change sets compare new template with old template. They ignore drift entirely.
- **Drift-aware change sets** (November 2025) compare template, last deployment and live state.

> ON THE EXAM
>
> Detect drift across many stacks on a schedule: the AWS Config rule, not a scheduled Lambda.

Classic change set, or drift-aware?

DRIFT-AWARE CHANGE SET

Three-way comparison

- Template, last deployment, and live state
- Shows drift the update will overwrite
- Can revert drifted properties to the template
- Deployment mode REVERT_DRIFT

CLASSIC CHANGE SET

Two-way comparison

- New template against old template
- Blind to manual changes
- Unchanged properties keep their drift
- Still the default

THE TIEBREAKER

'Restore resources changed by hand to the template's configuration, and preview it first': a drift-aware change set.

Bootstrapping an instance from the template

- **AWS::CloudFormation::Init** metadata declares packages, files, users, commands and services.
- **cfn-init**, called from user data, reads that metadata and applies it.
- **configSets** control the order in which groups of configuration run.
- Declarative metadata is easier to read and update than a long user data script.

> ON THE EXAM
>
> The helper scripts ship with Amazon Linux AMIs. Instances need a network path to the CloudFormation endpoint.

Waiting for the instance to say it is ready

```
1  WebServerGroup:
2    Type: AWS::AutoScaling::AutoScalingGroup
3    CreationPolicy:
4      ResourceSignal: { Count: 3, Timeout: PT15M }
5    # launch template user data runs:
6    #   /opt/aws/bin/cfn-init -s ${AWS::StackName} -r LaunchTemplate
7    #   /opt/aws/bin/cfn-signal -e $? -s ${AWS::StackName} -r WebServerGroup
8    Properties:
9      MinSize: 3
10     MaxSize: 6
```

4 Wait for three success signals, for at most fifteen minutes.

7 Signal cfn-init's exit code: success only if configuration worked.

ON THE EXAM

No signals in time, or a failure signal: the resource fails and the stack rolls back.

cfn-hup: configuration changes without replacement

- **cfn-hup** is a daemon that checks the metadata for changes, every 15 minutes by default.
- When metadata changes, its hooks run actions, usually cfn-init again.
- So a template update can change files or services on running instances, in place.
- Without cfn-hup, changing Init metadata does nothing to instances already running.

> ON THE EXAM
>
> Metadata updated, running instances unchanged: cfn-hup is missing or not configured.

Update policies for Auto Scaling groups

UPDATEPOLICY	WHAT AN UPDATE DOES	ROLLBACK
`AutoScalingRollingUpdate`	Replaces instances in batches, keeping a minimum in service	Rolls batches back
`AutoScalingReplacingUpdate`	Creates a whole new group, then removes the old	Keeps the old group: fast
`AutoScalingScheduledAction`	Stops the template resetting sizes set by schedules	Not applicable

ON THE EXAM

Rolling update with WaitOnResourceSignals makes each batch prove itself with cfn-signal before the next.

When a stack operation fails

- By default a failed operation **rolls back** to the last good state.
- **Rollback triggers** name CloudWatch alarms, watched during and up to 180 minutes after an update.
- Choosing to **preserve successfully provisioned resources** keeps them for debugging a failed create or update.
- UPDATE_ROLLBACK_FAILED: fix the cause, then **continue update rollback**, skipping resources if needed.

> ON THE EXAM
>
> Roll back a stack update if application errors rise shortly afterwards: rollback triggers on the stack.

Custom resources

- A **ServiceToken** points at a Lambda function or SNS topic that handles Create, Update and Delete.
- The handler must send its result to a **presigned S3 URL**. That is how CloudFormation hears back.
- **ServiceTimeout** (June 2024), 1 to 3600 seconds, replaces the old fixed wait of one hour.
- A handler that ignores Delete, or cannot reach S3, leaves the stack waiting.

> ON THE EXAM
>
> Stack stuck for an hour on a custom resource whose function succeeded: the response never reached the S3 URL.

Getting values into templates safely

- **Dynamic references** resolve at deploy time: resolve:ssm, resolve:ssm-secure, resolve:secretsmanager.
- The secret never appears in the template, the parameters, or the stack's console view.
- **NoEcho** parameters hide a value in output, but someone still has to supply it.
- A plaintext default value for a password parameter is visible to anyone who reads the template.

> ON THE EXAM
>
> Database password in a template, least exposure: a Secrets Manager dynamic reference, or a managed master password.

Nested stacks, or cross-stack references?

NESTED STACKS

One unit, reusable parts

- A parent stack contains child stacks
- Deployed and rolled back together
- Good for reusable components
- Outputs passed parent to child

EXPORTS AND IMPORTVALUE

Separate lifecycles

- One stack exports, others import
- Each stack deployed on its own
- Exporter cannot delete an export in use
- Nor change its value while imported

THE TIEBREAKER

Network owned by one team, apps by others: exports. Components always deployed together: nested stacks.

Reshaping what already exists

SITUATION	TOOL
Resources exist, but no stack manages them	Resource import, with DeletionPolicy set
Many existing resources, and no template	IaC generator: scan, then generate a template or CDK app
Resources in the wrong stack, or badly named	Stack refactoring (February 2025)
Old way to move a resource	Retain, remove from stack, import elsewhere

ON THE EXAM

Split a monolithic stack without recreating resources: stack refactoring, previewed, then executed atomically.

A template creates an Auto Scaling group in private subnets with no NAT gateway, using a CreationPolicy that waits for three signals within 15 minutes. User data runs cfn-init and then cfn-signal.

Stack creation fails after 15 minutes with 'received 0 SUCCESS signals'. Connecting to an instance shows the application installed and running correctly.

What should the DevOps engineer do?

A Increase the signal timeout to one hour

B Remove the CreationPolicy from the Auto Scaling group

C Give the instances a path to the CloudFormation endpoint, such as an interface VPC endpoint

D Replace the CreationPolicy with a WaitCondition

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A **Longer timeout**

The signals are not slow. They can never arrive, so any timeout expires.

B **Remove the CreationPolicy**

Stops the failure by removing the readiness check. The stack no longer knows the fleet works.

C **Endpoint path for cfn-signal**

The application works; the signal cannot leave the subnet. A CloudFormation interface endpoint carries it.

D **WaitCondition**

Its signals also need to reach AWS. Same network problem, older mechanism.

THE SIGNAL

Software fine, zero signals, private subnets: cfn-signal cannot reach CloudFormation.

A developer changed a property on the production RDS instance in a stack's template. The change required replacement, and the update created a new, empty database.

The company wants to prevent any stack update, by anyone or any pipeline, from replacing or deleting that database, with the least operational overhead.

What should the DevOps engineer implement?

A Set DeletionPolicy: Retain on the database resource

B An IAM policy that denies developers rds:DeleteDBInstance

C Enable termination protection on the stack

D A stack policy that denies Update:Replace and Update:Delete on the database resource

◆ PAUSE AND DECIDE

What should the DevOps engineer implement?

A DeletionPolicy: Retain

Applies when the resource leaves the stack. A replacement still happens, with a new empty database.

B IAM deny for developers

CloudFormation or the pipeline performs the replacement, not the developer directly.

C Termination protection

Blocks deleting the whole stack. Updates are unaffected.

D Stack policy on the database

Blocks the update itself, for every user and pipeline, until deliberately overridden.

THE SIGNAL

'Prevent any update from replacing' is a stack policy. Add UpdateReplacePolicy: Snapshot as a second safety net.

During an incident, engineers edited several security group rules and an S3 bucket policy by hand in the console. Those resources belong to a CloudFormation stack.

The team wants the next deployment to return every hand-edited property to the template's values, and to preview that before it happens, with the least effort.

What should the team do?

A Create a drift-aware change set that reverts drift, review it, and execute it

B Delete the stack and create it again from the template

C Run drift detection, then reverse each difference by hand in the console

D Enable the Config rule for stack drift so the team is alerted next time

◆ PAUSE AND DECIDE

What should the team do?

A Drift-aware change set

Previews the drift it will overwrite, then restores template values, in one reviewed operation.

B Delete and recreate

Destroys resources, including anything stateful, to fix configuration. Far too much.

C Detect, then fix by hand

Works, but more manual console edits: the very thing that caused the drift.

D Config drift rule

Detects future drift. Fixes nothing that has already happened.

THE SIGNAL

Restore hand edits to the template, with a preview: a drift-aware change set. A classic one would leave them.

> A stack includes a Lambda-backed custom resource. The function runs in private subnets with no NAT gateway and no VPC endpoints, because it configures a private database.
>
> The stack stays in CREATE_IN_PROGRESS for an hour and then fails. The function's logs show it configured the database successfully within seconds.

How should the DevOps engineer fix this?

A Increase the Lambda function's timeout to 15 minutes

B Add an S3 gateway endpoint so the function can send its response, and set a shorter ServiceTimeout

C Change the custom resource to be backed by an SNS topic

D Add a WaitCondition that depends on the custom resource

◆ PAUSE AND DECIDE

How should the DevOps engineer fix this?

A Longer Lambda timeout

The work finished in seconds. Time was never the problem.

B S3 gateway endpoint, shorter ServiceTimeout

The response goes to a presigned S3 URL, which this subnet cannot reach. A shorter timeout fails fast next time.

C SNS-backed

The handler would still need to write its response to S3.

D WaitCondition

Adds another wait. The custom resource still never responds.

THE SIGNAL

Work succeeded, stack waited an hour: the response never reached S3. Same shape as a missing cfn-signal.

Module 9 in five lines

- Change sets preview; drift-aware change sets also see and revert hand edits.
- DeletionPolicy for leaving the stack; UpdateReplacePolicy for replacement; stack policy to forbid either.
- Creation policy plus cfn-signal proves software is ready; cfn-hup applies later metadata changes.
- Custom resources and signals must reach AWS endpoints, or the stack waits.
- Dynamic references for secrets. Import, generator and refactoring reshape existing estates.

> ON THE EXAM
>
> Next, module 10: StackSets, modules, Service Catalog, and Hooks with Guard.